

JS Promises and Async / Await

1. What is a Promise?

- a. A *Promise* in JavaScript is an object that represents the eventual *completion* or *failure* of an asynchronous operation and its resulting value.
- b. The Promise Object represents the completion or failure of an asynchronous operation and its results.

2. States of a Promise:

- a. *Pending*: The promise is in its initial state, neither fulfilled nor rejected.
- b. *Fulfilled*: The promise is successfully completed, and the resulting value is available.
- c. *Rejected*: The promise failed, and an error or reason is available.
- d. *Settled*: A promise is said to be settled when it is either *fulfilled* or *rejected*, but not pending anymore.

3. Why do we need promises in JS?

- a. Better Handling of Asynchronous Code
- b. Error Handling
- c. Avoid Callback Hell

4. Creating a Promise:

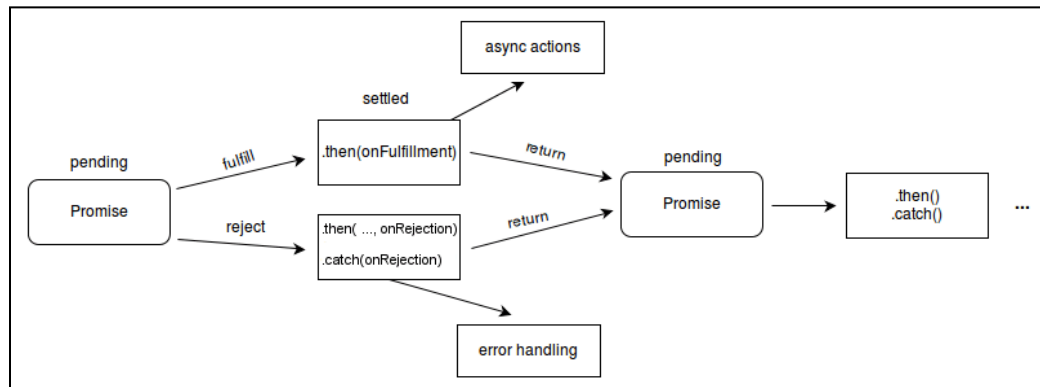
a. You can create a promise using the *Promise()* constructor.

b. **Syntax** →

```
let myPromise = new Promise((resolve, reject) => {  
  // Perform some asynchronous task  
  // Call resolve() if successful  
  // Call reject() if there's an error  
});
```

c. *resolve(value)* is a function that represents the promise is fulfilled.

d. *reject(error)* is a function that represents the promise is rejected.



e. **For Example** →

```
let myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if (success) {  
    resolve("Promise is fulfilled");  
  } else {  
    reject("Promise is rejected");  
  }  
});  
console.log(myPromise);  
//Promise {<fulfilled>: 'Promise is fulfilled'}
```

5. How to handle the promise?

- a. There are two ways to handle the promises in JavaScript are as follows:
 - i. By using `.then()` and `.catch()`
 - ii. By using `async` and `await`

6. Using `.then()` and `.catch()`

- a. Creating Promise:

```
let myPromise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Promise is fulfilled");
    } else {
        reject("Promise is rejected");
    }
});
```

- b. Now we will handle the promise, if the promise is resolved or rejected by using `.then()` and `.catch()` methods.

```
myPromise.then((result) => {
    console.log("Success:", result);
})
.catch((error) => {
    console.log("Error:", error);
})
.finally(() => {
    console.log("Promise is settled");
});
```

c. **Promise Instance Methods:**

- i. **then()**: Handles the fulfilled (success) state.
- ii. **catch()**: Handles the rejected (failure) state.
- iii. **finally()**: Executes code no matter what (fulfilled or rejected).

d. **Final Output:**

```
Success: Promise is fulfilled
Promise is settled
```

7. What are async and await?

a. **async** and **await** are **syntactic sugar** built on top of Promises.

b. They make asynchronous code look **synchronous** and **cleaner**.

c. **async**:

- i. The **async** keyword is used to declare a function that always returns a Promise.

d. **await**:

- i. The **await** keyword makes the function pause execution until the promise is resolved (fulfilled or rejected).
- ii. It can only be used inside an **async** function.

e. **Rules:**

- i. **async** keyword is used with functions.
- ii. An **async** function always returns a Promise.
- iii. **await** is used **inside async functions** to pause execution until the Promise is settled.

8. Handling Promise by using **async** and **await**

a. Same example:

```
let myPromise = new Promise((resolve, reject) =>
{
    let success = true;

    if (success) {
        resolve("Promise is fulfilled");
    } else {
        reject("Promise is rejected");
    }
});
```

b. Handling promise by using **async** and **await** keyword:

c. For handling errors while handling promises we have to use **try** and **catch** blocks.

```
async function handlePromise() {
    try {
        let result = await myPromise;
        console.log("Success:", result);
    } catch (error) {
        console.log("Error:", error);
    } finally {
        console.log("Promise is settled");
    }
}

handlePromise();
```

9. Built-in Promise Methods:

a. There are 6 built-in methods of promise are there:

- i. `Promise.all()`
- ii. `Promise.race()`
- iii. `Promise.any()`
- iv. `Promise.allSettled()`
- v. `Promise.resolve()`
- vi. `Promise.reject()`

10. `Promise.all()`:

a. The `Promise.all()` method returns a single promise that resolves only when *all promises are successful*, and rejects immediately if *any one promise fails*.

b. **Syntax** →



```
Promise.any([p1, p2, p3]).then((results) => { console.log(results);  
}).catch((error) => { console.log(error); });
```

c. **For Example** →

```
1 let p1 = new Promise((resolve) => setTimeout(() => {resolve("first resolved")}, 1000));  
2 let p2 = new Promise((resolve) => setTimeout(() => {resolve("second resolved")}, 2000));  
3 let p3 = new Promise((resolve) => setTimeout(() => {resolve("third resolved")}, 3000));  
4 let p4 = new Promise((resolve, reject) => setTimeout(() => {reject("fourth rejected")}, 4000));  
5  
6 // It will return all the resolved promises in the form of array.  
7 Promise.all([p1,p2,p3]).then((results) => {console.log(results)})
```

d. **Output:** If we pass [p1,p2,p3,p4] we will get the output like

✖ ▶ Uncaught (in promise) fourth rejected

e. If any promise is rejected , then it also returns the resolved array and error for the rejected promise.

f. Final Output:

```
▶ (3) ['first resolved', 'second resolved', 'third resolved']  
✖ ▶ Uncaught (in promise) fourth rejected
```

11. Promise.race():

- a. The **Promise.race()** method returns a promise that settles with the result of *the first promise that finishes*, whether it is resolved or rejected.

b. Syntax →



```
Promise.race([p1, p2, p3]).then((results) => { console.log(results);  
}).catch((error) => { console.log(error); });
```

c. For Example →

```
1 let p1 = new Promise((resolve) => setTimeout(() => {resolve("first resolved")}, 1000));  
2 let p2 = new Promise((resolve) => setTimeout(() => {resolve("second resolved")}, 2000));  
3 let p3 = new Promise((resolve) => setTimeout(() => {resolve("third resolved")}, 3000));  
4 let p4 = new Promise((resolve, reject) => setTimeout(() => {reject("fourth rejected")}, 4000));  
5  
6  
7 Promise.race([p1,p2,p3,p4]).then((results) => {console.log(results)})
```

d. Output:

```
first resolved
```

- e. If there are rejected promises comes first then also **Promise.race()** returns that promise because it is a settled promise.

12. `Promise.any()`:

- a. The `Promise.any()` method returns a promise that resolves as soon as *one promise is successful*, and rejects only if *all promises fail*.
- b. **Syntax** →



```
Promise.any([p1, p2, p3]).then((results) => { console.log(results);  
}).catch((error) => { console.log(error); });
```

- c. **For Example** →

```
1 let p1 = new Promise((resolve, reject) => setTimeout(() => {reject("first rejected")}, 1000));  
2 let p2 = new Promise((resolve, reject) => setTimeout(() => {reject("second rejected")}, 2000));  
3 let p3 = new Promise((resolve) => setTimeout(() => {resolve("third resolved")}, 3000));  
4 let p4 = new Promise((resolve, reject) => setTimeout(() => {reject("fourth rejected")}, 4000));  
5  
6 // It returns the resolved promise.  
7 Promise.any([p1, p2, p3, p4]).then((results) => {console.log(results)})
```

- d. **Output:**

```
third resolved
```

13. `Promise.allSettled()`:

- a. The `Promise.allSettled()` method returns a promise that resolves after *all promises have finished*, giving the result of each (success or failure).
- b. Once all are settled, it returns an array with the results of all promises, including both successes and failures.
- c. This method never rejects, even if some promises fail. It always gives you all the results, no matter what happened to each promise.

d. **Syntax** →



```
Promise.allSettled([p1, p2, p3]).then((results) => {  
  console.log(results); }).catch((error) => { console.log(error); });
```

e. **For Example** →



```
1 let p1 = new Promise((resolve, reject) => setTimeout(() => {resolve("first resolved")}, 1000));  
2 let p2 = new Promise((resolve, reject) => setTimeout(() => {reject("second rejected")}, 2000));  
3 let p3 = new Promise((resolve) => setTimeout(() => {resolve("third resolved")}, 3000));  
4 let p4 = new Promise((resolve, reject) => setTimeout(() => {reject("fourth rejected")}, 4000));  
5  
6 // It returns the resolved promise.  
7 Promise.allSettled([p1, p2, p3, p4]).then((results) => {console.log(results)})
```

f. **Final Output:**

```
▼ (4) [{...}, {...}, {...}, {...}] ⓘ  
  ▶ 0: {status: 'fulfilled', value: 'first resolved'}  
  ▶ 1: {status: 'rejected', reason: 'second rejected'}  
  ▶ 2: {status: 'fulfilled', value: 'third resolved'}  
  ▶ 3: {status: 'rejected', reason: 'fourth rejected'}  
    length: 4  
  ▶ [[Prototype]]: Array(0)
```

14. **Promise.resolve():**

a. The **Promise.resolve()** method creates a promise that is *already resolved* with the given value.

b. **Syntax** →



```
Promise.resolve("Success").then((value) => { console.log(value);  
});
```

c. **For Example** →



```
1 let promise = Promise.resolve("Success");
2 promise.then((result) => {console.log(result)});
3 // Output: Success
```

15. `Promise.reject()`:

a. The `Promise.reject()` method creates a promise that is *already rejected* with the given error or reason.

b. **Syntax** →



```
Promise.reject("Error occurred").catch((error) => {
  console.log(error); });
```

c. **For Example** →



```
1 let promise = Promise.reject("Error Occured");
2 promise.catch((error) => {console.log(error)});
3 // Output: Error Occured
```

16. Real-Life Applications of Promise Methods:

1. **Promise.all()** → When you need all tasks to finish together.

👉 **Example:** Fetching weather, news, and stock data at the same time.

2. **Promise.race()** → When you only care about the fastest result.

👉 **Example:** Picking the quickest server response from many servers.

3. **Promise.allSettled()** → When you want to know the result of every task (success or fail).

👉 **Example:** Uploading multiple files and checking which ones uploaded and which failed.

4. **Promise.any()** → When you need just one success, no matter which one.

👉 **Example:** Using the first API/server that gives a successful response if others fail.

5. **Promise.resolve()** → When you want to create a promise that is already successful.

👉 **Example:** Returning mock success data immediately during testing.

6. **Promise.reject()** → When you want to create a promise that is already failed.

👉 **Example:** Rejecting a promise if form validation fails (like wrong user input).